System Calls

Linux System Programming

Dr R K Singla
Computer Science & Applications
Panjab University, Chandigarh

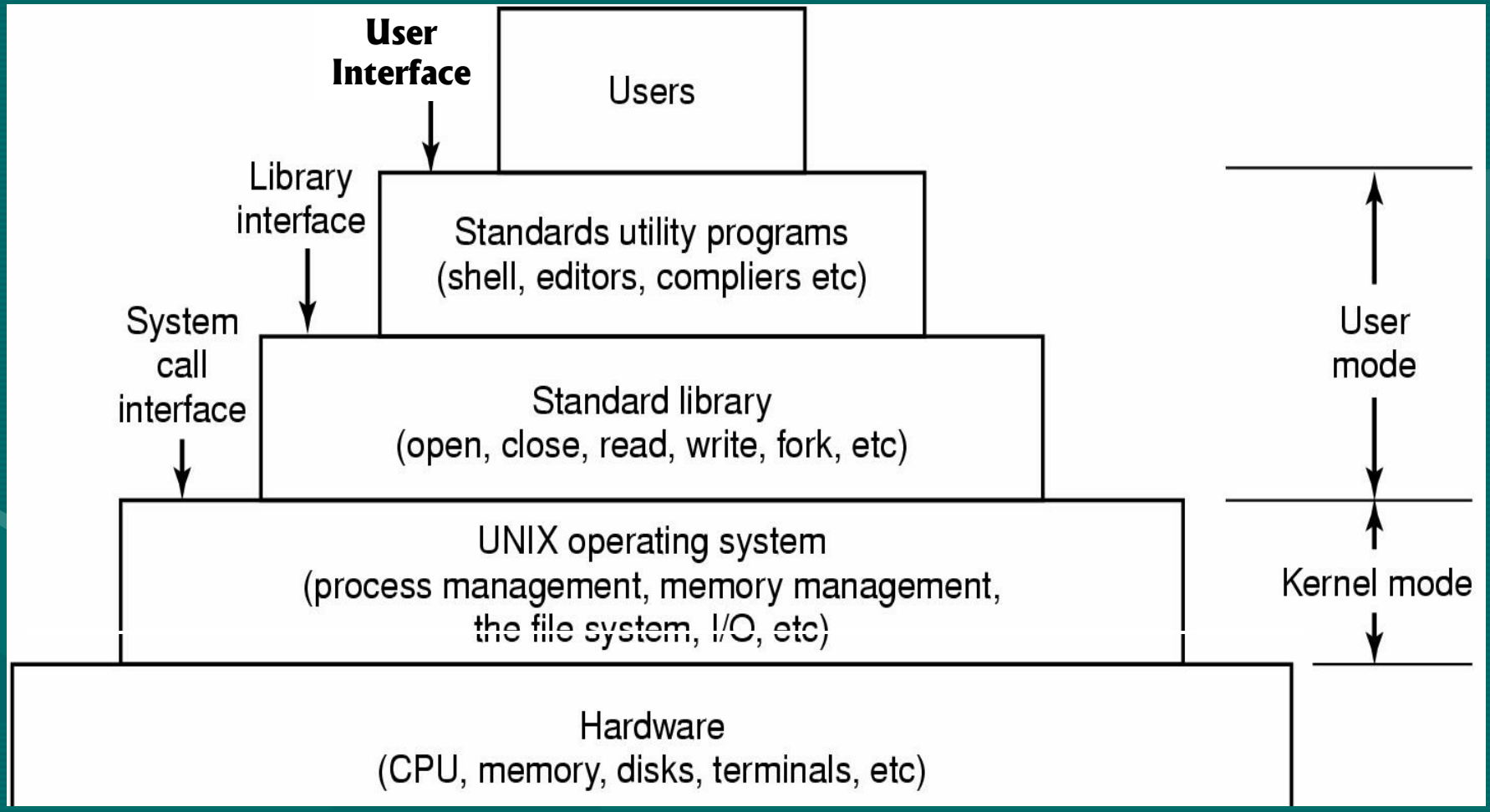
The Linux System: Design Principles

- Linux is a multiuser, multitasking system with a full set of UNIX-compatible tools.
- Its file system adheres to traditional UNIX semantics, and it fully implements the standard UNIX networking model.
- Main design goals are speed, efficiency, flexibility and standardization.
- Linux is designed to be compliant with the relevant POSIX documents; some Linux distributions have achieved official POSIX certification.
- The Linux programming interface adheres to the SVR4 UNIX semantics, rather than to BSD behavior.

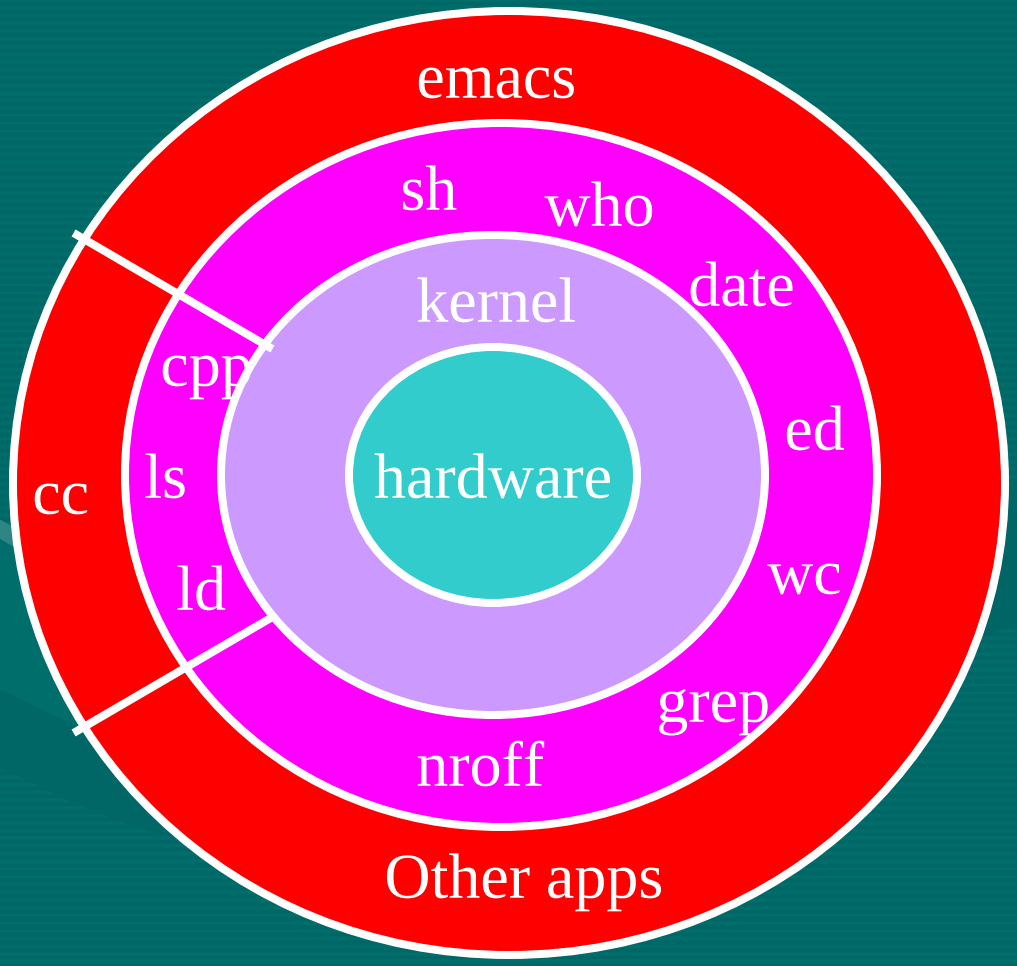
Linux: Introduction

- Operating System: a system that manages the resources of a computer.
- Resources: CPUs, Memory, I/O devices, Network
- Kernel: the memory resident portion of Linux system
- File system and process control system are two major components of Linux Kernel.

The layers of a Linux system



Architecture of Linux System



- ✓OS interacts directly with the hardware
- ✓Such OS is called system kernel

Components of a Linux System

- Like most UNIX implementations, Linux is composed of three main bodies of code:
  - Standard Library
  - Kernel
  - Standard Utilities Programs
- Standard Utilities Programs perform individual specialized management tasks.
  - Shell
  - Commands for the management of files and directories
  - Filters
  - Compilers
  - Editors
  - Commands for the administration of the system
  - …

The shell of a Linux System

- The UNIX systems have a Graphical User Interface (Linux uses KDE, GNOME …), but the programmers prefer to type the commands.
- **Shell:** the user process which executes programs (command interpreter)
  - **User types command**
  - **Shell reads command (read from input) and translates it to the operating system.**
- Can run *external programs* (e.g. firefox) or *internal shell commands* (e.g. **cd**)
- Various different shells available:
  - **Bourne shell (sh), C shell (csh), Korn shell (ksh), TC shell (tcsh), Bourne Again shell (bash).**
- The administrator of the system provides to the user a default shell, but the user can change shell.

The shell of a Linux System

- Example of commands
  - $ cd /usr/src/linux
  - $ more COPYING
  - $ cp file1 file2
  - $ head –20 file
  - $ head 20 file
  - $ ls *.c
  - $ ls [abc]*
  - $ sort < file1 > file2
- String multiple commands together in a shell script
  - $ sort < file1 > file2; head –30 < file2 ; rm temp
  - $ sort < file | head –30
  - $ grep org CREDITS | sort | head –20 | tail –5 > file

The shell is more ...

- The shell is programmable, that is it possible to make the shell scripts
- A shell script is a file containing a sequence of commands addressed to the operating system that facilitates the repeated execution of the included commands without their having to be laboriously retyped each time they are executed.
- If there is a distinct ordered list of operating system commands that the user needs to execute repeatedly, for example, immediately after every login or immediately before every logout, then most operating systems have a facility for recording the list of commands in a file, which can then either be executed automatically upon login or logout, or can be invoked by the user through the issuance of a single command that results in the execution of the entire contents of the batch file, which can contain as few as one operating system command or as many as thousands.

Linux Utility Programs

Program	Typical use
cat	Concatenate multiple files to standard output
chmod	Change file protection mode
cp	Copy one or more files
cut	Cut columns of text from a file
grep	Search a file for some pattern
head	Extract the first lines of a file
ls	List directory
make	Compile files to build a binary
mkdir	Make a directory
od	Octal dump a file
paste	Paste columns of text into a file
pr	Format a file for printing
rm	Remove one or more files
rmdir	Remove a directory
sort	Sort a file of lines alphabetically
tail	Extract the last lines of a file
tr	Translate between character sets

A few of the more common Linux utility programs required by POSIX

Linux System Kernel

- Three major tasks of kernel:
  - Process Management
  - Device Management
  - File Management
- Three additional Services for Kernel:
  - Virtual Memory
  - Networking
  - Network File Systems
- Experimental Kernel Features:
  - Multiprocessor support
  - Lightweight process (thread) support

The Kernel

- The kernel is the very heart of a Linux system. It controls the resources, memory, schedules processes and their access to CPU and is responsible of communication between software and hardware components.
- The kernel provides the lowest-level abstraction layer for the resources like memory, CPU and I/O devices. Applications that want to perform any function with these resources communicate with the kernel using system calls.
- System calls are generic functions (such as write) and they will handle the actual work with different devices, process management and memory management. The advantage in system calls is that the actual call stays the same regardless of the device or resource being used. Porting the software for different versions of the operating system also becomes easier when the system calls are persistent between versions.

The Kernel

- Kernel memory protection divides the virtual memory into *kernel space* and *user space*.
- Kernel space is reserved for the kernel, its' extensions and the device drivers.
- User space is the area in memory where all the user mode applications work. User mode application can access hardware devices, virtual memory, file management and other kernel services running in kernel space only by using system calls.
- There are over 100 system calls in Linux, documentation for those can be found from the Linux kernel system calls manual pages (man 2 syscalls).

The Kernel

- The kernel is responsible for maintaining the important abstractions of the operating system.
- It provides the main functions of the abstract machine (system calls and Interrupt and traps).

Approximate structure of generic Linux kernel

<table>
<tr><td colspan="3">System calls</td><td colspan="4">Interrupts and traps</td></tr>
<tr><td colspan="2">Terminal handing</td><td>Sockets</td><td>File naming</td><td>Map-ping</td><td>Page faults</td><td rowspan="2">Signal handling</td><td rowspan="2">Process creation and termination</td></tr>
<tr><td rowspan="2">Raw tty</td><td>Cooked tty</td><td>Network protocols</td><td>File systems</td><td colspan="2">Virtual memory</td></tr>
<tr><td>Line disciplines</td><td>Routing</td><td>Buffer cache</td><td colspan="2">Page cache</td><td colspan="2">Process scheduling</td></tr>
<tr><td colspan="2">Character devices</td><td>Network device drivers</td><td colspan="2">Disk device drivers</td><td colspan="2">Process dispatching</td></tr>
<tr><td colspan="7">Hardware</td></tr>
</table>

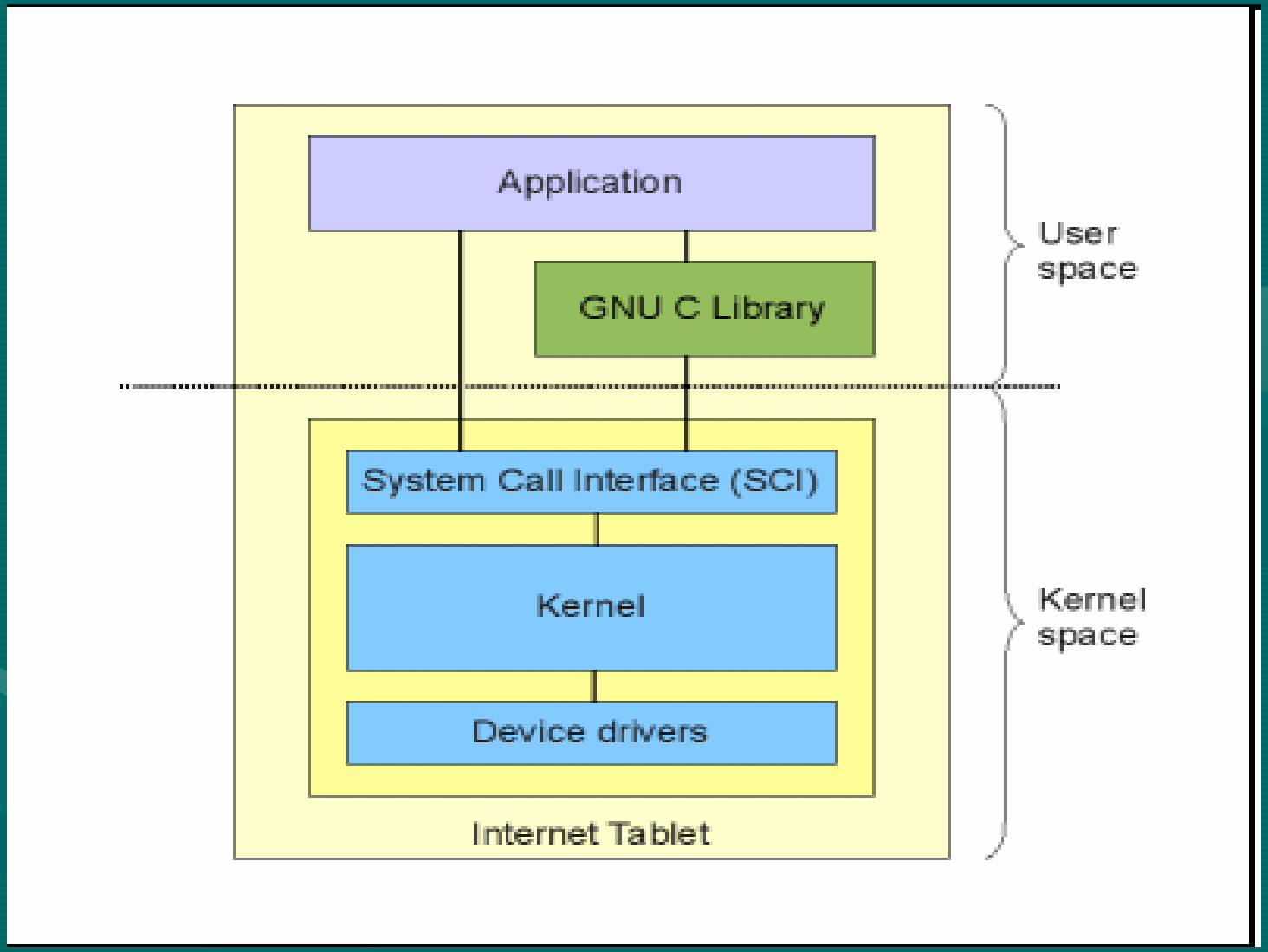
The Kernel

- The Linux kernel uses a monolithic model. It does not take the "new" client-server model (i.e., micro-kernel). The main reason for this choice is the improvement of the performances.
  - The kernel code is executed in kernel mode with full access to all the physical resources of the computer.
  - All kernel code and data are contained in a unique address space.
- Although the kernel runs as a single process with a single address space, Linux kernel uses the modularity.

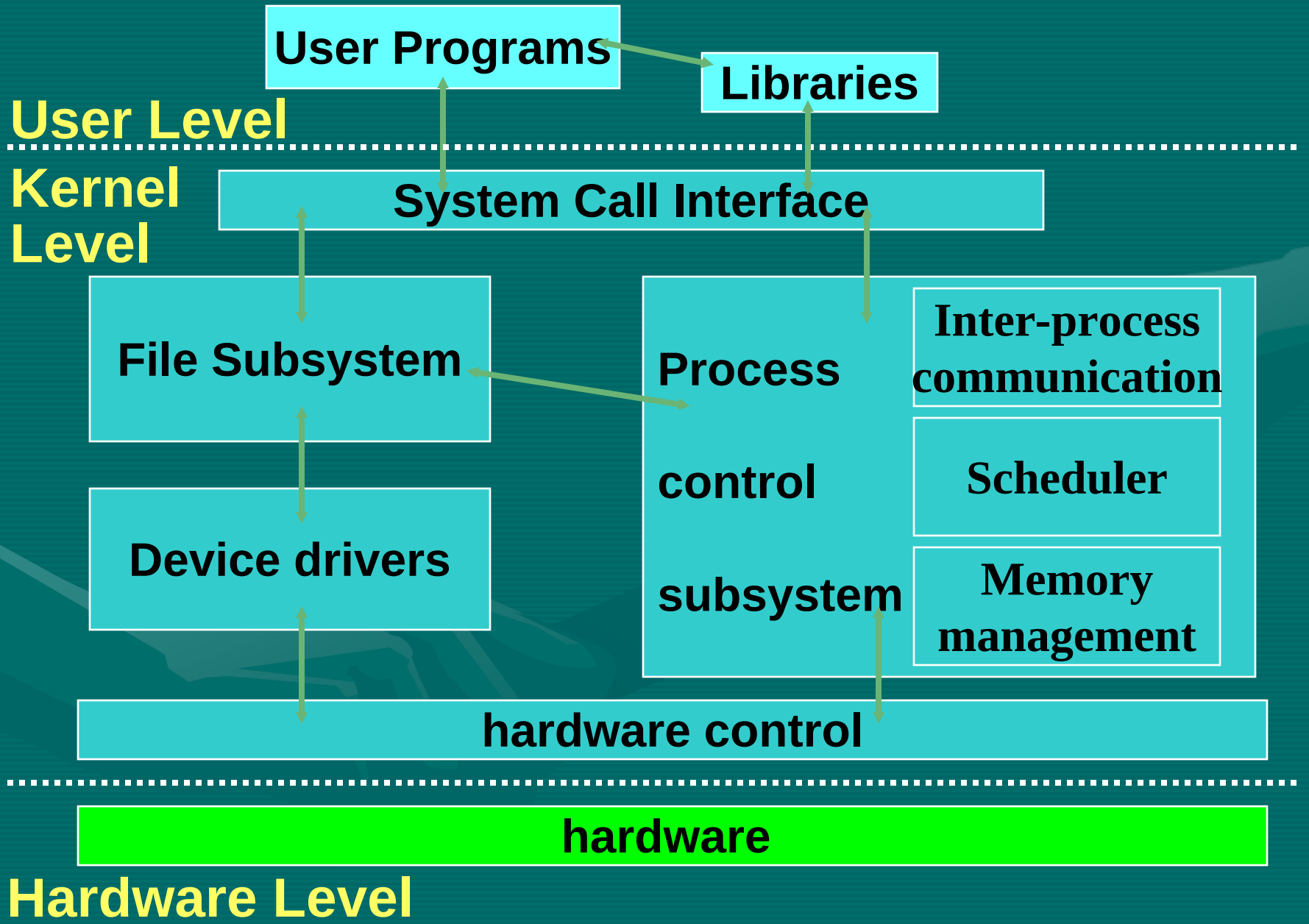
The Kernel

- Kernel architecture, where the kernel is run in kernel space in supervisor mode and provides system calls to implement the operating system services is called **monolithic** kernel.
- Most modern monolithic kernels, such as Linux, provides a way to dynamically load and unload executable kernel modules at runtime. The modules allow extending the kernel's capabilities (for example adding a device driver) as required without rebooting or rebuilding the whole kernel image.
- In contrast, **microkernel** is an architecture where device drivers and other code are loaded and executed on demand and are not necessarily always in memory.

The Kernel, Hardware & Software relations



Block Diagram of System Kernel



Process Control Subsystem

- Process Synchronization
- Inter-process communication
- Memory management:
- Scheduler: process scheduling
  (allocate CPU to Processes)

File subsystem

- A file system is a collection of files and directories on a disk or tape in standard Linux file system format.
- Kernel’s file syb-system regulates data flow between the kernel and secondary storage devices.

Hardware Control

- Hardware control is responsible for handling interrupts and for communicating with the machine.
- Devices such as disks or terminals may interrupt the CPU while a process is executing.
- The kernel may resume execution of the interrupted process after servicing the interrupt.

What are Processes?

- A program is an executable file.
- A process is an instance of the program in execution.
- For example: create two active processes

```
$ emacs &
$ emacs &
$ ps
PID    TTY   TIME CMD
12893  pts/4 0:00 tcsh
12581  pts/4 0:01 emacs
12582  pts/4 0:01 emacs
$
```

What are Processes?

- A process has
  - text: machine instructions
    (may be shared by other processes)
  - data
  - stack
- Process may execute either in user mode and in kernel mode.
- Process information are stored in two places:
  - Process table
  - User table

What are Processes?

- A process is a program that is being executed. It consists of the executable program code, a bunch of resources (for example open files), an address space, internal data for kernel, possibly threads (one or more) and a data section.
- Each process in Linux has a series of characteristics associated with it, below is a (far from complete) list of available data:
  - PID: Process ID, numeric identifier of the process
  - State: State of the process (running, stopped etc.)
  - PPID: Parent process ID
  - Children
  - Open files: List of open files for the process
  - TTY: The terminal to which the process is connected
  - RUID: Real user ID, the owner of the process
- Every process has a PID, process ID. This is a unique identification number used to refer to the process and a way to the system to tell processes apart from each other. When user starts the program, the process itself and all processes started by that process will be owned by that user (process RUID), thus processes' permissions to access files and system resources are determined by using permissions for that user

Process management

- The Process table is managed by the Kernel.
- It holds one "line" per process; c.f ps –l shows some of these … status, uid, pid, ppid whether in ram or swapped out and pointers to a "***per process data area***". …this holds saved registers (if swapped out) and users ids for file access. These tables are not accessible by a user - they are held in kernel memory space. The user data region is where the user's program is stored (data and instructions or just data).

User data region

- Each user program requires data space… a user stack and data area (heap). And a code space. A process image is comprised of the ***PPDA***, the user stack, user data and code. It gets swapped to disk.
- Code can be read only… allows more than one user to run it at the same time. Each user has his own user data but code may be shared read only called ***"pure text"*** in Unix
- Read only code need not be swapped (written) to disk, since it hasn't changed. Unix has a third table to keep track of read-only shared code .

User mode and Kernel Mode

- A process will execute instructions in the user program until some outside facility is required or an outside event, such as a clock interrupt , occurs. The CPU switches from ***user mode*** to ***kernel mode***.
- When the execution of kernel instructions is complete another process will resume. The kernel has a ***process schedule*** to look after this. A process may be suspended or blocked (waiting on an external event such as disk I/O to complete)

User mode and Kernel mode

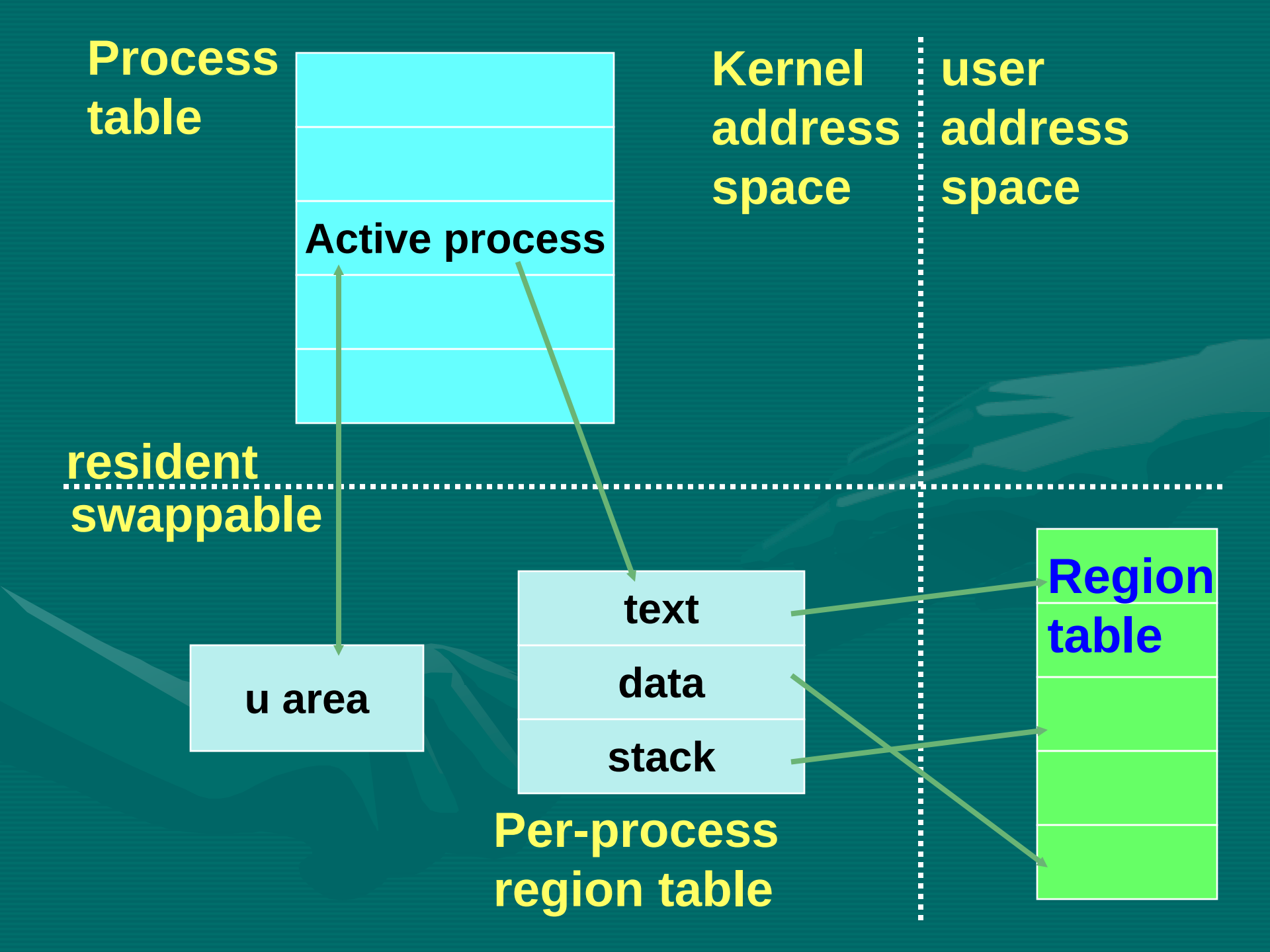
- At any given instant a computer running the Linux system is either executing a process or the kernel itself is running
- The computer is in user mode when it is executing instructions in a user process and it is in kernel mode when it is executing instructions in the kernel.
- Executing System call ==> User mode to Kernel mode
  - ✓ perform I/O operations
  - ✓ system clock interrupt

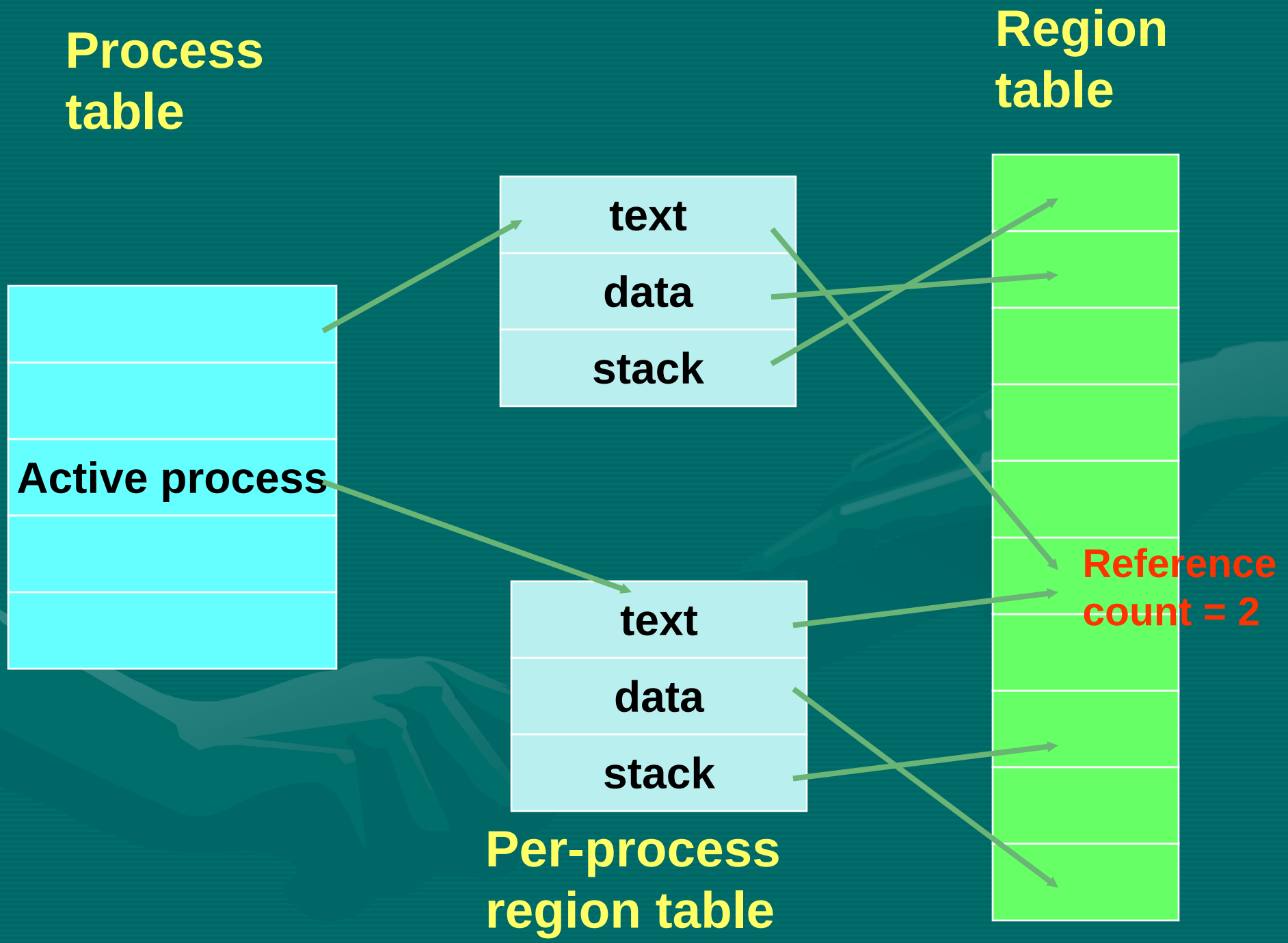
Process Table

- Process table: an entry in process table has the following information:
  - ✓process state:
    - A. running in user mode or kernel mode
    - B. Ready in memory or Ready but swapped
    - C. Sleep in memory or sleep and swapped
  - ✓PID: process id
  - ✓UID: user id
  - ✓scheduling information
  - ✓signals that is sent to the process but not yet handled
  - ✓a pointer to per-process-region table
- There is a single process table for the entire system

User Table (u area)

- Each process has only one private user table.
- User table contains information that must be accessible while the process is in execution.
  - ✓A pointer to the process table slot
  - ✓parameters of the current system call, return values error codes
  - ✓file descriptors for all open files
  - ✓current directory and current root
  - ✓process and file size limits.
- User table is an extension of the process table.


Process table
Active process
Kernel address space
user address space
resident
swappable
u area
text
data
stack
Per-process region table
Region table


Process table
Region table
text
data
stack
Active process
text
data
stack
Reference count = 2
Per-process region table

Shared Program Text and Software Libraries

- Many programs, such as shell, are often being executed by several users simultaneously.
- The text (program) part can be shared.
- In order to be shared, a program must be compiled using a special option that arranges the process image so that the variable part (data and stack) and the fixed part (text) are cleanly separated.
- An extension to the idea of sharing text is sharing libraries.
- Without shared libraries, all the executing programs contain their own copies.

Difference between Trap and Interrupt

- A trap is an exception in a user process. It's caused by division by zero or invalid memory access. It's also the usual way to invoke a kernel routine (a system call) because those run with a higher priority than user code. Handling is synchronous (so the user code is suspended and continues afterwards).
- An interrupt is something generated by the hardware (devices like the hard disk, graphics card, I/O ports, etc). These are asynchronous (i.e. they don't happen at predictable places in the user code).
- A trap is a special kind of interrupt which is commonly referred to as a software interrupt. An interrupt is a more general term which covers both hardware interrupts (interrupts from hardware devices) and software interrupts (interrupts from software, such as traps).

The Standard Library

- A system call is the method that the user process uses to ask for an action from the O.S.
- The programs perform the system calls by mean of **trap**.

  trap instruction:
  - changes from *user mode* to *kernel mode*
  - controls the correctness of the call parameters
  - execution done on behalf of the operating system
  - returns to user mode
- Since it is impossible to write a trap in C, it is provided a **standard library** with a procedure for each **system call**. These procedures are written in assembler and are called from C. For example a C program for performing the system call `read`, it calls the procedure *read* ,of the standard library.
- So, the **standard library** defines a standard set of functions through which applications interact with the kernel, and which implement much of the operating-system functionality that does not need the full privileges of kernel code.
- **POSIX** establishes which are the procedures of the library that the system has to provide, their parameters and tasks.

System Call

- Operating systems offer processes running in User Mode ***a set of interfaces*** to interact with ***hardware devices*** such as
  - the **CPU**
  - disks

    and
  - printers.
- **Unix** systems implement most interfaces between *User Mode processes* and *hardware devices* by means of ***system calls*** issued to the kernel.

System Call

- A process accesses system resources through system call.
- System call for
  - Process Control:

    fork: create a new process

    wait: allow a parent process to synchronize its execution with the exit of a child process.

    exec: invoke a new program.

    exit: terminate process execution
  - File system:

    File: open, read, write, lseek, close

    inode: chdir, chown chmod, stat fstat

    others: pipe dup, mount, unmount, link, unlink

Linux System Calls

* System Calls

- A system call is a direct entry point through which an active process can obtain services from the kernel.

- The system calls are specific routines in the operating system kernel that are directly accessible to application programs.

Features of Linux System Calls

* Features of System Calls
  - The system calls allow you to perform low-level I/O, manipulate files and directories, create and control multiple concurrent processes, manage interrupts, and control I/O to terminals.
  - Because UNIX is implemented in C, its system calls are specified in C syntax and directly called from C programs.
  - A system call may need one or more associated header files. These header files are clearly indicated with each call described.

POSIX APIs vs. System Calls

- An *application programmer interface* is a ***function definition*** that specifies how to obtain a given service.
- A *system call* is an explicit request to the kernel made via a *software interrupt*.

From a Wrapper Routine to a System Call

- Linux systems include several *libraries of functions* that provide **API**s to programmers.
- Some of the **API**s defined by the **`libc`** standard **C** library refer to *wrapper routines* (routines whose only purpose is to issue a *system call*).
- Usually, each system call has a corresponding wrapper routine, which defines the **API** that application programs should employ.

APIs and System Calls

- An **API** does not necessarily correspond to a specific system call.
  - First of all, the **API** could offer its services directly in User Mode. (For something abstract such as math functions, there may be no reason to make system calls.)
  - Second, a single **API** function could make several system calls.
  - Moreover, several **API** functions could make the same system call, but wrap extra functionality around it.

Example of Different APIs Issuing the Same System Call

- In Linux, the **`malloc( )`**, **`calloc( )`**, and **`free( )`** APIs are implemented in the **`libc`** library.
- The code in this library keeps track of the allocation and deallocation requests and uses the **`brk( )`** system call to enlarge or shrink the *process heap*.

The Return Value of a Wrapper Routine

- Most wrapper routines return an integer value, whose meaning depends on the corresponding system call.
- A return value of **-1** usually indicates that the ***kernel*** was unable to satisfy the process request.
- A failure in the ***system call handler*** may be caused by
  - invalid parameters
  - a lack of available resources
  - hardware problems, and so on.
- The specific ***error code*** is contained in the **`errno`** variable, which is defined in the **`libc`** library.

Execution Flow of a System Call

- When a ***User Mode process*** invokes a ***system call***, the **CPU** switches to **Kernel Mode** and starts the execution of a ***kernel function***.
  - As we will see in the next section, in the 80x86 architecture a Linux system call can be invoked in two different ways.
- The net result of both methods, however, is a jump to an assembly language function called the ***system call handler***.

System Call Number

- Because the kernel implements many different system calls, the User Mode process must pass a parameter called the *system call number* to identify the required system call.
- The **eax** register is used by Linux for this purpose.
  - As we'll see in the section "Parameter Passing" later in this chapter, additional parameters are usually passed when invoking a system call.

The Return Value of a System Call

- All system calls return an integer value.
- The conventions for these return values are different from those for wrapper routines.
  - In the kernel
    - positive or 0 values denote a successful termination of the system call
    - negative values denote an error condition
      - In the latter case, the value is the negation of the error code that must be returned to the application program in the `errno` variable.
  - The `errno` variable is not set or used by the kernel. Instead, the wrapper routines handle the task of setting this variable after a return from a system call.

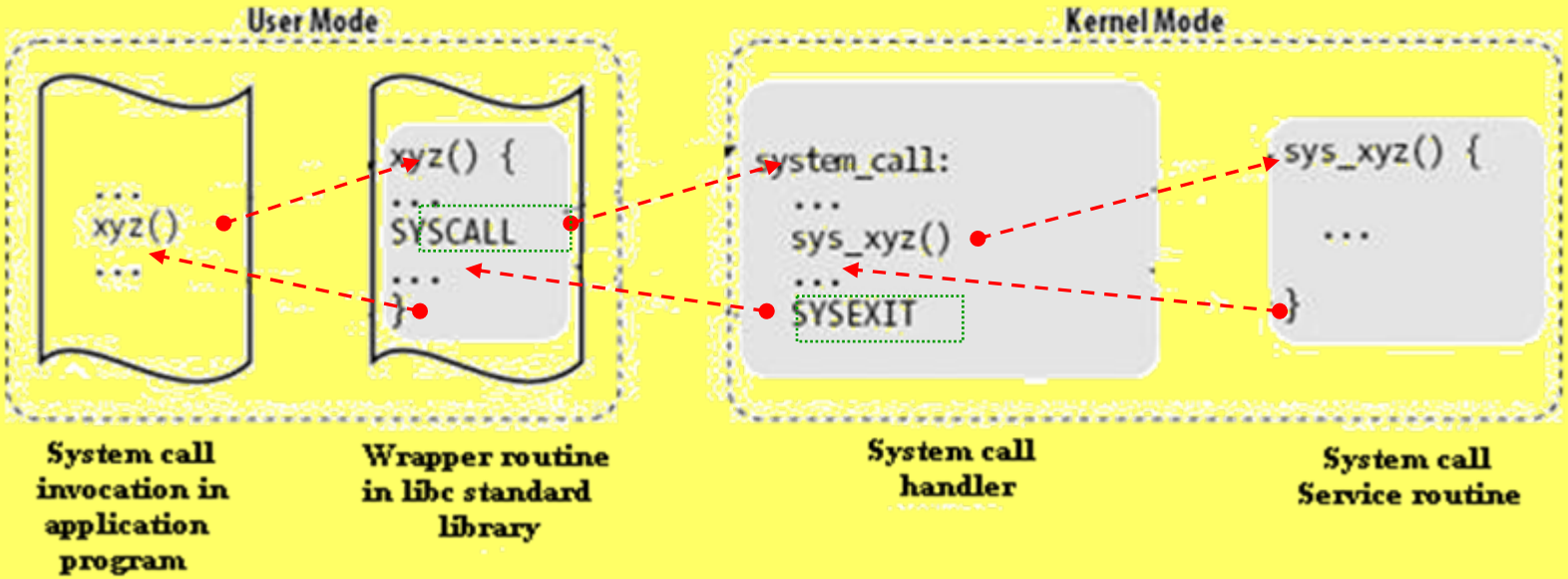
Operations Performed by a System Call

- The *system call handler*, which has a structure similar to that of the other *exception handlers*, performs the following operations:
  - Saves the contents of most registers in the Kernel Mode stack.
    - This operation is common to all system calls and is coded in assembly language.
  - Handles the system call by invoking a corresponding **C** function called the *system call service routine*.
  - Exits from the handler:
    - the registers are loaded with the values saved in the Kernel Mode stack
    - the **CPU** is switched back from Kernel Mode to User Mode.
      - This operation is common to all system calls and is coded in assembly language.

Naming Rules of System Call Service Routines

- The name of the service routine associated with the ***xyz( )*** system call is usually **sys_*xyz*( )**; there are, however, a few exceptions to this rule.

Control Flow Diagram of a System Call



- The **arrows** denote the execution flow between the functions.
- The terms "**SYSCALL**" and "**SYSEXIT**" are placeholders for the actual assembly language instructions that switch the **CPU**, respectively, from User Mode to Kernel Mode and from Kernel Mode to User Mode.

Content

1. Introduction
2. Unix Processes
3. Creating a new Process: `fork()`
4. Terminating a process: `exit()`
5. `wait()` and `waitpid()`

Context

Context

$$\left\{ \begin{array}{l} \text{- single computer (single CPU)} \\ \text{- under UNIX operating system} \end{array} \right.$$

- It is possible to have several simultaneously executing programs
  $\longrightarrow$ It appears that the executions are done in Parallel!
- Multiprogramming:
  The CPU executes some instructions of one program then switches to another program giving the illusion that any program is continuously executing, this scheme is called *time-slicing*.

Program vs. process

- A *program* is an executable file residing on a disk.
- A *process* is an executing (running) program, usually with a limited life-time.
  Note that sometimes a process is called *task*.

Important notion

An executing program $\longrightarrow$ a *process*.

Unix Processes

Every process in Unix contains the following:

- A unique process ID (PID): a unique nonnegative integer assigned by Unix, used to identify a process.
- The user ID of the owner.
- Some code: instructions that are being executed (the code segment)
- Some data: variables (the data segment)
- A stack: a form of memory where it is possible to push and pop instructions (the stack segment).
- An environment: registers' contents, tables of open files,...

getpid() and getppid()

getpid() and getppid()

A process can obtain its PID by the `getpid()` system call.
A process can also obtain its parent ID PPID by the `getppid()` system call.

```
#include <stdio.h>

int main(void){

  printf("Hello, my PID is %d\n", getpid());
  printf("Hello, my PPID is %d\n", getppid());
  exit(0);
}

Shell-Prompt> a.out
Hello, my PID is 11723 Hello, my PPID is 5598
```

States of a process

Different states of a process

Throughout its lifetime, a process can have the following states:

- **New**: the process has just been created.
- **Ready**: waiting to be assigned to a processor (the processor is busy running other processes).
- **Running**: while its instructions are being executed.
- **Blocked**: waiting for an event (e.g. I/O).
- **Done**: finished

- A new process becomes ready and waits for its turn to be executed by the processor.
- A ready process becomes running for a "time slice" after which it is either done, blocked or ready to continue for another time slice.
- A blocked process becomes ready again after the event it was waiting for occurs (e.g. I/O).

init

init

- Unix starts as a single process, called `init`. The `PID` of `init` is 1.
- The only way to create a new process in Unix, is to duplicate an existing one.
  → the process `init` is the ancestor of all subsequent processes.
- In particular, process `init` never dies.

Creation of a new process

The creation or spawning of new processes is done with two system calls:

- `fork()`: duplicates the caller process
- `exec()`: replaces the caller process by a new one.

Example 1: Handling a login session

- `init` forks a copy of itself.
- the child process `exec`s the `getty` program which prints the login prompt, waits for the user's login name, and then `exec`s the `login` program.
- the `login` process, if successful, `exec`s the shell program (e.g. `csh`).

Example 2: Running a utility from a shell (`csh`).

- `csh` forks a copy of itself.
- the child process `exec`s the utility program, for example `netscape`.
- the parent process waits for the termination(`exit`) of its child process by going asleep.
- when the child process terminates, a signal is sent to the parent process (the shell program `csh`). The latter wakes-up and becomes ready to accept the next command.

Creating a new Process: `fork()`

`fork()`

Synopsis: `pid_t fork(void);`

when successful, the `fork()` system call:

- creates a copy of the caller (parent) process.
- returns the `PID` of the newly created process to the parent
- returns 0 to the new process (the child).

If not successful, the `fork()` returns -1.

`fork()` is a strange system call: called by a single process but returns twice, to two different processes.

The child process

A child process has:

- its own unique `PID`,
- a different `PPID`,
- its own copy of the parent's data segment and file descriptors.

Both parent and children resume the execution after the call to `fork()`.

`fork()` is primarily used in two situations:

1. A process wants to execute another program (Shells).
2. A process has a main task and when necessary creates a child to handle an operation (Servers).

both parent and children resume the execution after the call to `fork()`

```
#include <unistd.h>

int main(int argc, char *argv[]){
  int pid;

  printf("Only one process\n");
  pid = fork();

  if(pid == -1){
    perror("impossible to fork");
    exit(1);
  }
  if(pid > 0)
    printf("I am the parent, pid=%d\n", getpid());
  else
    if(pid == 0)
      printf("I am the child, pid=%d\n", getpid());

  exit(0);
}
```

The child has its own copy of the parent's data segment

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int glb = 100; // global variable
int main(){
   int pid;
   int var = 88;
   printf("Before fork\n");
   pid = fork();
   if( pid < 0 ){
      perror("fork");
      exit(1);
   }
   if(pid == 0){ //child
      glb++; var++;
   }
   else{ // parent
        sleep(2);
   }
   printf("pid = %d, glob = %d, var = %d\n", getpid(), glb, var);
   return 0;
}
```

The child has its own copy of the parent's file descriptors and share the same offset

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#include <stdlib.h>

int main(){
   int pid, fd, i;
   char c;

   if ( (fd =open("test",O_RDWR|O_CREAT, 0644)) == -1 ){
     perror("test");
   }

   if( (pid = fork())< 0 ){
     perror("fork");
     exit(1);
   }
   if(pid == 0) // child
     for( i =65; i < 85; i++){
       c = i; write(fd, &c ,1);
   }
   else // parent
   for( i = 0 ; i< 20; i++){
      c = 64; // character @
      write(fd, &c ,1);
   }
return 0;
}
```

What is the output?

```
int main(){
  printf("before fork, my pid is %d\n" , getpid());
  fork();
  fork();
  fork();
  printf("done, my pid is %d\n", getpid());
}
```

```
int main(){
  int i;
  printf("before fork, my pid is %d\n" , getpid());
  for (i=0; i<3; i++){
    if ( fork()== 0 )
      printf("Hi, I am child. My pid is %d\n", getpid());
  }
}
```

Process termination: `exit()`

Synopsis: `void exit(int status);`

- This call terminates a process and never returns
- The `status` value is available to the parent process through the `wait()` system call.

When invoked by a process, the `exit()` system call:

- closes all the process's file descriptors
- frees the memory used by its code, data and stack
- sends a **SIGCHLD** signal to its parent and waits for the parent to accept its return code.

Example

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/types.h>
#include <sys/wait.h>

int main() {
  int newpid;
  printf("before: mypid is %d\n", getpid());
  if ((newpid = fork()) == -1 )
        perror("fork");
  else if (newpid == 0){
         printf("I am the child %d now sleeping...\n",getpid());
         sleep(2);
         exit(47);
         printf("I am gone");
       }
       else{
         printf("I am the parent %d\n",getpid());
         sleep(10);
         printf("My child %d must be gone by now. I am leaving...\n",newpid);
         exit(1);
         printf("I am gone too\n");
       }
}
```

What is the output produced?

The output of the previous program is:

```
before:  mypid is 5067
I am the parent 5067
I am the child 5068 now sleeping...
My child 5068 must be gone by now.  I am
leaving...
```

The lines `printf("I am gone");` and `printf("I am gone too");` are not executed because `exit()` never returns.

wait() and waitpid()

wait()

Synopsis: `pid_t wait(int *status);`
Allows the parent to wait for the termination of one of its children and accept its termination code.

waitpid()

Synopsis: `pid_t waitpid(pid_t pid, int *status, int options);`
`waitpid()` is similar to `wait()` except that it has a number of options that control which process it wait for.
For example:

- If `pid` > 0 then, `waitpid()` waits for the child process identified by `pid`.with
- If `pid` = -1 then, `waitpid()` waits for any child process to return.

Termination

When successful, `wait()` returns the `pid` of the terminating child process.
The value in `status` is encoded as follow:

- if the rightmost byte of `status` is zero, then the leftmost byte contains the status returned by the child: a value between 0 and 255 (passed as an argument to exit). This represents a normal termination of the child process.
- if the rightmost byte of `status` is nonzero, then the rightmost 7 bits are equal to the `signal number`, that caused the process to terminate. The remaining bit of the rightmost byte is set to 1 if a core dump was produced by the child process.

Example

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/types.h>
#include <sys/wait.h>

int main() {
  int newpid;
  printf("before: mypid is %d\n", getpid());
  if ((newpid = fork()) == -1 )
        perror("fork");
  else if (newpid == 0){
          printf("I am the child %d now sleeping...\n",getpid());
          sleep(5);
          exit(47);
        }
        else{
          printf("I am the parent %d\n",getpid());
          int status;
          int child_pid  = wait(&status);
          printf("My child %d has terminated\n",child_pid);

          printf("I have received the status = %d\n",status);

          int child_status = status >> 8;
          int signal = status & 0x7F;
          int core = status & 0x80;

          printf("Child status = %d Signal = %d Core = %d\n",child_status, signal, core);
        }
}
```

Example of a normal termination

Example

Consider the previous program. The output resulting from a normal execution:

```
before:  mypid is 4602
I am the parent 4602
I am the child 4603 now sleeping...
My child 4603 has terminated
I have received the status = 12032
Child status = 47 Signal = 0 Core = 0
```

Example of a premature termination

Example

1. Consider the previous program running. When the child is sleeping, the output is as follows:

```
before:  mypid is 4602
I am the parent 4602
I am the child 4603 now sleeping...
```

2. Before the child makes the `exit` call, we `kill` the child process. At the command prompt, we run:

```
kill 4603
```

3. The parent wakes up and displays:

```
My child 4603 has terminated
I have received the status = 15
Child status = 0 Signal = 15 Core = 0
```